

Ngày: __ / __ / __



MACHINE LEARNING

• GHI CHÚ CƠ BẢN •

Từ Cơ Bản Đến Xây Dựng Mô Hình Thông Minh



DỮ LIỆU

Thu thập & Tìm hiểu



THUẬT TOÁN

Học & Áp dụng



MÔ HÌNH

Huấn luyện & Dự đoán



ĐÁNH GIÁ

Đo lường & Cải thiện



TRIỂN KHAI

Áp dụng thực tế



NỘI DUNG CHÍNH

- Giới thiệu về Machine Learning
- Các loại Machine Learning
- Các loại Data & Data Preprocessing
- Feature Engineering
- Train, Test, Split
- Các thuật toán (Linear Regression, Logistic Regression, KNN, v.v.)
- Các chỉ số đánh giá mô hình
- Overfitting & Underfitting
- Hyperparameter Tuning
- Ví dụ thực hành

“Machine Learning giúp máy tính có khả năng học hỏi mà không cần được lập trình tường minh.”



» BẠN SẼ HỌC ĐƯỢC GÌ«

- ☒ Hiểu rõ các khái niệm Machine Learning
- ☒ Áp dụng thuật toán vào dữ liệu thực tế
- ☒ Xây dựng & đánh giá mô hình ML
- ☒ Thực hiện bước đi đầu tiên vào thế giới AI

★ Học Hôm Nay. Xây Dựng Ngày Mai. Đổi Mới Mãi Mãi. ★

Machine Learning DÀNH CHO LẬP TRÌNH VIÊN

Kiến Thức Từ Cơ Bản Đến Trung Cấp

1. Machine Learning Là Gì?

Machine Learning (ML) là một nhánh của Artificial Intelligence (AI), giúp máy tính có thể học từ dữ liệu và tự cải thiện mà không cần được lập trình một cách tường minh.

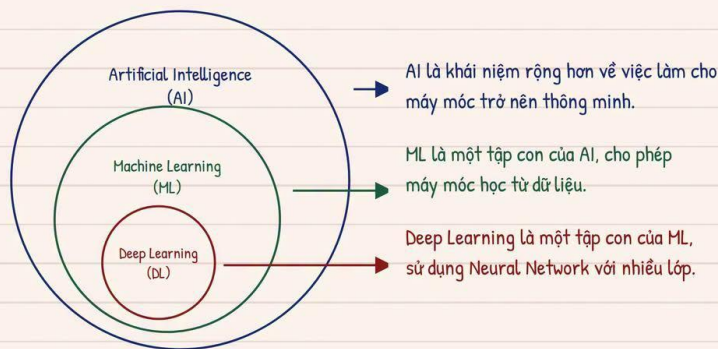
Nói một cách đơn giản:

"Học từ dữ liệu trong quá khứ để đưa ra quyết định hoặc dự đoán tốt hơn cho tương lai."

2. Ví Dụ Thực Tế

- Netflix gợi ý phim dựa trên lịch sử xem của bạn.
- Gmail tự động lọc email rác.
- Amazon gợi ý sản phẩm bạn có thể thích.
- Google Maps dự đoán tình trạng giao thông và chỉ ra tuyến đường tốt nhất.

3. AI → ML → Deep Learning

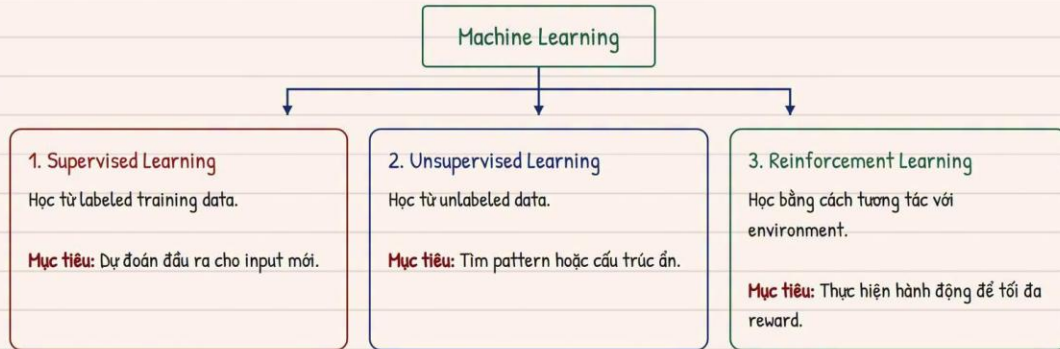


4. Lập Trình Truyền Thống so với Machine Learning

Lập Trình Truyền Thống	Machine Learning
• Bạn viết các quy tắc tường minh. • Máy tính tuân theo các quy tắc đó. • Ví dụ: if age > 18 → Đủ điều kiện • Phù hợp với bài toán đơn giản, rõ ràng.	• Bạn cung cấp dữ liệu. • Máy tính tự học quy luật từ dữ liệu. • Ví dụ: học quy luật đủ điều kiện từ dữ liệu quá khứ. • Phù hợp với bài toán phức tạp trong thực tế.

2. Các Loại Machine Learning

Machine Learning được chia thành ba loại chính, dựa trên loại dữ liệu và cách thức học.



Chi Tiết & Ví Dụ

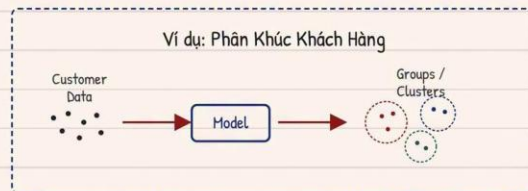
1. Supervised Learning

- Sử dụng labeled data (input + output).
- Model học ánh xạ từ input sang output.
- Loại: **Regression, Classification**



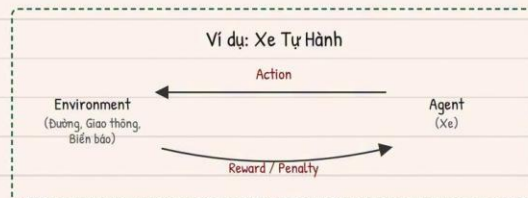
2. Unsupervised Learning

- Sử dụng unlabeled data (chỉ có input).
- Model tìm ra pattern hoặc nhóm dữ liệu.
- Loại: **Clustering, Dimensionality Reduction, Association**



3. Reinforcement Learning

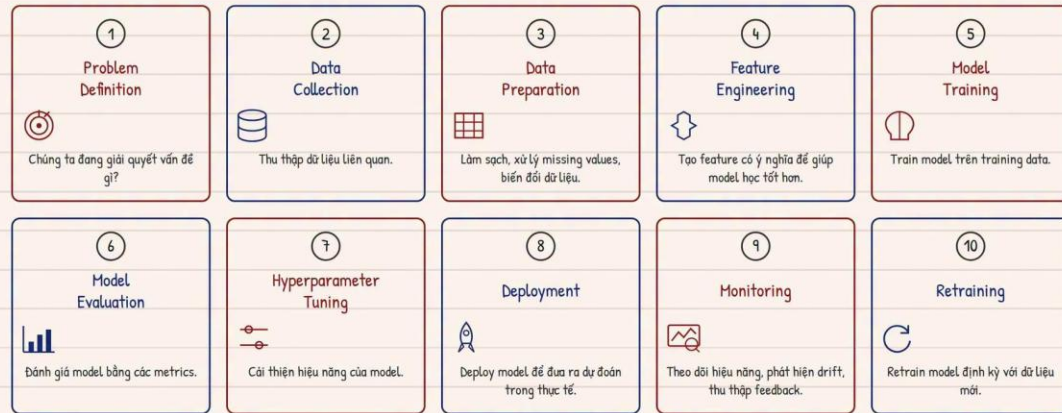
- Agent học thông qua thử và sai.
- Nhận reward cho hành động tốt, penalty cho hành động xấu.
- Mục tiêu: Tối đa hóa tổng reward.



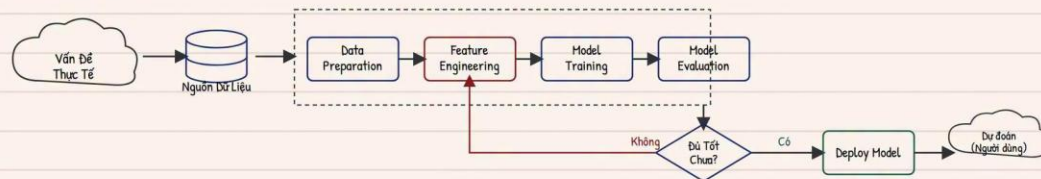
Hãy chọn đúng loại Machine Learning phù hợp dựa trên bài toán và dữ liệu bạn có.

3. Machine Learning Workflow

Machine Learning tuân theo một workflow từng bước, đi từ vấn đề đến dự đoán và cải thiện.



Workflow Diagram



Key Points

- Không phải bài toán nào cũng đi theo một đường thẳng.
- Bạn có thể lặp lại giữa các bước để cải thiện kết quả.
- Dữ liệu tốt + feature tốt = model tốt hơn.
- Monitoring và retraining giúp model duy trì độ chính xác theo thời gian.

Flow Summary

Problem → Data → Prepare → Features
→ Train → Evaluate → Tune → Deploy
→ Monitor → Retrain

4. Dataset, Features & Labels

Trong Machine Learning, chúng ta làm việc với data.
Hiểu về **dataset**, **features** và **labels** là bước đầu tiên để xây dựng bất kỳ ML model nào.

→ 1. Dataset

Dataset là một tập hợp các examples hoặc records.
Mỗi record chứa một số thông tin về một đối tượng.

Ví dụ: Dataset Giá Nhà

House ID	Diện tích (sq.ft)	Số phòng ngủ	Vị trí	Giá (₹)
101	1200	2	Mumbai	50,00,000
102	1500	3	Delhi	65,00,000
103	1800	3	Bangalore	80,00,000
104	2000	4	Mumbai	95,00,000
105	2500	4	Delhi	1,10,00,000

Mỗi dòng → một example
(một căn nhà)

Mỗi cột → một thông tin
nào đó về căn nhà

→ 2. Features

Features (hay input variables) là các cột mô tả đặc điểm của dữ liệu. Chúng được model sử dụng để học các pattern.

Trong ví dụ trên, features là:

- Diện tích (sq.ft)
- Số phòng ngủ
- Vị trí

Những yếu tố này giúp model hiểu về căn nhà và dự đoán giá.

→ 3. Label (Target)

Label (hay target variable) là thứ chúng ta muốn model dự đoán. Đó chính là output.

Trong ví dụ trên, label là:

- Giá (₹)

Model học từ features và cố gắng dự đoán label (giá nhà).

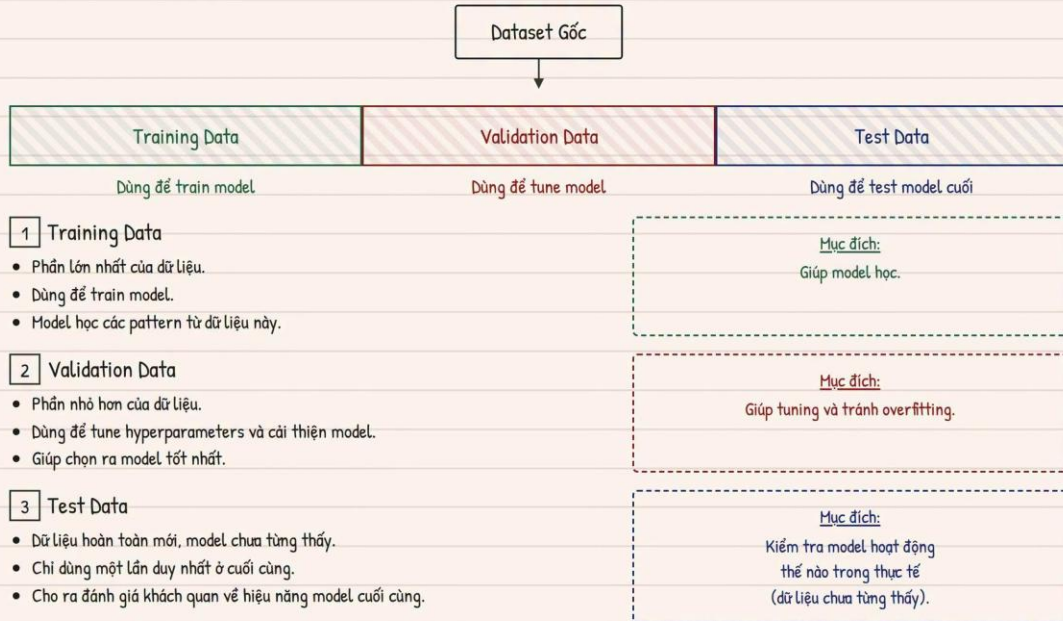
Tóm lại:

- Dataset → Toàn bộ dữ liệu
- Features → Đầu vào (biến độc lập)
- Label → Đầu ra (biến mục tiêu)

} Model sử dụng features để dự đoán label.

5. Training, Validation & Test Data

Để xây dựng một Machine Learning model tốt, chúng ta chia dataset thành **ba phần**.



Tỷ Lệ Chia Điển Hình (Ví dụ)

Training Data	Validation Data	Test Data
70%	15%	15%

Chú ý: · Tỷ lệ chính xác có thể thay đổi (ví dụ: 80-10-10 hoặc 70-15-15).
· Điều quan trọng là giữ test data hoàn toàn tách biệt và chưa từng được model thấy.

6. Data Preprocessing

Raw data thường lộn xộn và không phù hợp để đưa vào ML model.
Data Preprocessing giúp làm sạch và biến đổi dữ liệu để trở nên hữu ích cho việc training.

Tại sao nó quan trọng?

- Cải thiện độ chính xác của model
- Tăng tốc quá trình training
- Giúp xử lý dữ liệu thiếu và không nhất quán
- Giúp dữ liệu dễ hiểu hơn

Các Bước Preprocessing Phổ Biến

1 Handling Missing Values

- Điền giá trị thiếu hoặc xóa các dòng/cột có quá nhiều giá trị thiếu.

Ví dụ (Trước)				Sau		
Tuổi	Thu nhập	TP		Tuổi	Thu nhập	TP
25	50000	Delhi	→	25	50000	Delhi
—	60000	Mumbai		28	60000	Mumbai
30	—	Pune		30	55000	Pune

(Dùng Mean/Median để điền)

2 Removing Duplicates

- Xóa các dòng trùng lặp để tránh bias.

Trước		Sau
A, 25, Delhi	→	A, 25, Delhi
A, 25, Delhi		B, 30, Mumbai
B, 30, Mumbai		C, 22, Pune
C, 22, Pune		

3 Encoding Categorical Data

- Chuyển đổi categories thành số để model có thể hiểu được.

City → Delhi, Mumbai, Pune → City → 0, 1, 2
(Label Encoding / One-Hot Encoding)

4 Feature Scaling

- Đưa tất cả features về cùng một thang đo.
- Phương pháp phổ biến: **Standardization (Z-score)**, **Normalization (0 đến 1)**.

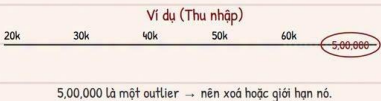
Trước khi Scaling
Tuổi: 10 – 70
Thu nhập: 20.000 – 2.00.000

↓

Sau khi Scaling (0 đến 1)
Tuổi: 0.0 – 1.0
Thu nhập: 0.0 – 1.0

5 Outlier Detection & Treatment

- Phát hiện các giá trị bất thường có thể ảnh hưởng xấu đến hiệu năng model.
- Xử lý bằng cách xóa, giới hạn (capping), hoặc biến đổi.



6 Feature Engineering

- Tạo ra các feature mới từ dữ liệu có sẵn.
- Giúp cải thiện độ chính xác của model.

Ngày sinh → Tuổi
Tổng = Số lượng × Đơn giá → Thành tiền

Tóm lại: Dữ liệu sạch → Học tốt hơn → Dự đoán tốt hơn

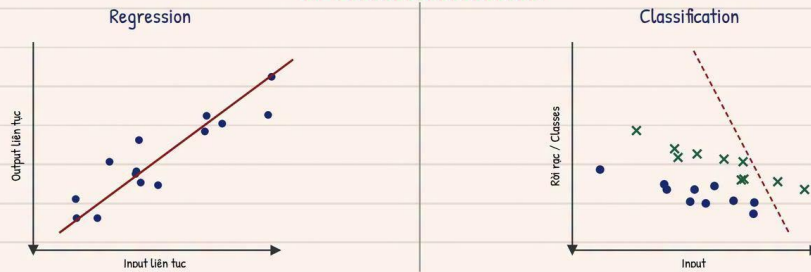
7. Regression

Regression là một loại supervised learning, trong đó model dự đoán các giá trị số liên tục (continuous numerical values).

Nó tìm ra mối quan hệ giữa input features và một continuous output.

Ví dụ: Dự đoán giá nhà, nhiệt độ, lương, doanh số, v.v.

Regression vs Classification



Các Thuật Toán Regression Phổ Biến

- Linear Regression
- Polynomial Regression
- Ridge Regression (L2 Regularization)
- Lasso Regression (L1 Regularization)
- Elastic Net Regression

Linear Regression (Simple)

Tìm đường thẳng phù hợp nhất: $y = mx + c$

trong đó, y = giá trị dự đoán

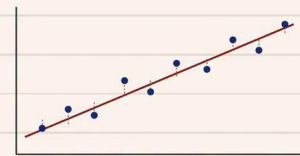
x = input feature

m = slope (độ thay đổi của y theo x)

c = intercept (giá trị y khi $x = 0$)

Cách Linear Regression Hoạt Động

- Cho một dataset gồm các cặp (x, y) .
- Model tìm đường thẳng ($y = mx + c$) sao cho error là nhỏ nhất.
- Error được đo bằng các metric như **Mean Squared Error**.
- Đường thẳng có error nhỏ nhất được xem là best fit.



Khoảng cách dọc từ điểm đến đường thẳng = error (residual)

Evaluation Metrics (Regression)

1. MAE

Mean Absolute Error
Trung bình các giá trị
tuyệt đối của sai số.

2. MSE / RMSE

Phạt nặng hơn với sai số
lớn. RMSE là căn bậc
hai của MSE.

3. R^2 Score

Cho biết model giải
thích được bao nhiêu
variance trong dữ liệu (0-1).

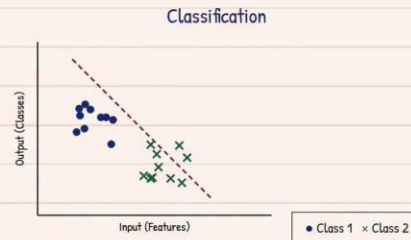
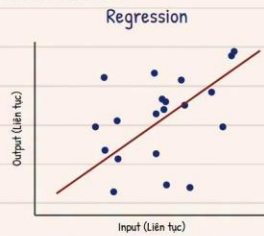
Regression được dùng khi câu trả lời là một con số, không phải một category.

8. Classification

Classification là một loại **supervised learning**, trong đó model dự đoán các **discrete classes** hoặc categories. Output là một class label, không phải một giá trị liên tục.

Ví dụ: Phát hiện spam, phân loại email, chẩn đoán bệnh, nhận diện hình ảnh, v.v.

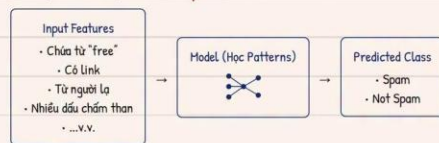
Regression vs Classification



Các Thuật Toán Classification Phổ Biến

- Logistic Regression
- K-Nearest Neighbors (KNN)
- Support Vector Machine (SVM)
- Decision Tree
- Random Forest
- Naive Bayes
- Gradient Boosting (XGBoost, LightGBM)

Cách Classification Hoạt Động (ví dụ: Phát hiện Email Spam)



Model học các pattern từ labeled data và dự đoán class cho dữ liệu mới.

Evaluation Metrics (Classification)

- Accuracy = $(TP + TN) / \text{Total}$
- Precision = $TP / (TP + FP)$
- Recall (Sensitivity) = $TP / (TP + FN)$
- F1 Score = $2 \times \text{Precision} \times \text{Recall} / (\text{Precision} + \text{Recall})$

TP = True Positive FP = False Positive

TN = True Negative FN = False Negative

Confusion Matrix

		Thực tế	
		Positive	Negative
Dự đoán	Positive	TP	FP
	Negative	FN	TN

Khi Nào Dùng Classification?

- Khi target variable là một category hoặc class.
- Khi câu trả lời là một label (ví dụ: Yes/No, Spam/Not Spam, Cat/Dog).

9. Clustering

Clustering là một kỹ thuật **unsupervised learning**.

Nó nhóm các data points tương tự nhau lại dựa trên pattern,

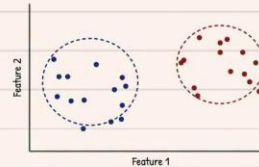
mà không cần bất kỳ label định sẵn nào.

Ví dụ: Phân khúc khách hàng, nhóm tài liệu, phân vùng hình ảnh, phát hiện bất thường.

Clustering Là Gì?

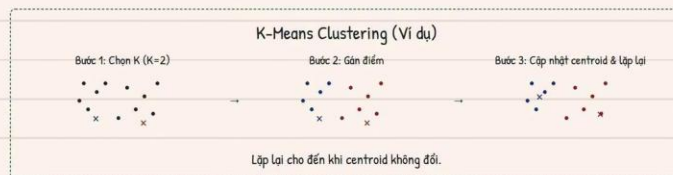
- Nhóm dữ liệu thành các cluster (hay segment).
- Dữ liệu trong cùng một cluster thì giống nhau hơn.
- Dữ liệu ở các cluster khác nhau thì khác biệt hơn.

Ví dụ Clustering (2 Clusters)



Các Thuật Toán Clustering Phổ Biến

1. K-Means Clustering
2. Hierarchical Clustering
3. DBSCAN (Density-Based Spatial Clustering of Applications with Noise)
4. Gaussian Mixture Models (GMM)



Ứng Dụng

- Phân khúc khách hàng
- Hệ thống gợi ý
- Phân tích giỏ hàng
- Nhóm tài liệu
- Phát hiện bất thường

Cách Clustering Hoạt Động (Các Bước Chung)

1. Chọn số lượng cluster (K).
2. Khởi tạo centroid (ngẫu nhiên hoặc theo một phương pháp).
3. Gán mỗi data point vào centroid gần nhất.
4. Cập nhật centroid dựa trên các điểm đã gán.
5. Lặp lại bước 3 và 4 cho đến khi hội tụ.

Các Loại Dữ Liệu

- Dữ liệu số → Dùng distance measure (ví dụ: Euclidean distance)
- Dữ liệu categorical → Dùng similarity measure (ví dụ: Jaccard, Hamming)
- Dữ liệu hỗn hợp → Dùng distance function đặc biệt hoặc encoding

Distance Measures (Phổ Biến)

- **Euclidean Distance:** Dùng cho dữ liệu số
- **Manhattan Distance:** Khoảng cách kiểu ô bàn cờ
- **Cosine Similarity:** Dùng cho dữ liệu nhiều chiều

Tóm Lại:

Clustering tìm ra các nhóm tự nhiên trong dữ liệu mà không cần label nào. Nó giúp phát hiện các pattern và cấu trúc ẩn.

10. Các Thuật Toán ML Quan Trọng

Các thuật toán khác nhau được dùng cho các loại bài toán khác nhau. Chọn đúng thuật toán là điều quan trọng để có hiệu năng model tốt. Dưới đây là một số thuật toán ML phổ biến nhất.

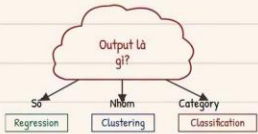
Thuật Toán	Loại	Ứng Dụng / Chức Năng	Điểm Chính
1. Linear Regression	Regression	- Dự đoán giá trị liên tục - Tìm đường best fit	- Đơn giản và nhanh - Giả định quan hệ tuyến tính
2. Logistic Regression	Classification	- Dự đoán kết quả nhị phân - Xuất ra xác suất (0 đến 1)	- Dùng hàm sigmoid - Phù hợp cho binary classification
3. Decision Tree	Classification / Regression	- Chia dữ liệu dựa trên điều kiện - Tạo cấu trúc dạng cây	- Dễ hiểu - Có thể overfit nếu cây quá sâu
4. Random Forest	Classification / Regression	- Ensemble của nhiều decision tree - Tăng accuracy & giảm overfitting	- Hoạt động tốt trên dataset lớn - Xử lý tốt dữ liệu phi tuyến tính
5. Support Vector Machine (SVM)	Classification / Regression	- Tìm ranh giới tốt nhất (hyperplane) - Hoạt động tốt trong không gian nhiều chiều	- Hiệu quả với dataset nhỏ - Kernel trick cho dữ liệu phi tuyến tính
6. K-Nearest Neighbors (KNN)	Classification / Regression	- Phân loại dựa trên các neighbor gần nhất - Đơn giản & học theo instance	- Không có giai đoạn training - Chậm trên dataset lớn
7. Naive Bayes	Classification	- Bộ phân loại xác suất - Dựa trên định lý Bayes	- Nhanh và hiệu quả - Hoạt động tốt trên dữ liệu văn bản
8. K-Means Clustering	Clustering	- Nhóm dữ liệu thành K cluster - Unsupervised learning	- Đơn giản & hiệu quả - Cần biết trước giá trị K

Tóm Tắt Theo Loại Thuật Toán

Thuật Toán Regression (Dự đoán số)	→	Linear Regression, Decision Tree Regression, Random Forest Regression, SVR, v.v.
Thuật Toán Classification (Dự đoán class)	→	Logistic Regression, Decision Tree, Random Forest, SVM, KNN, Naive Bayes, v.v.
Thuật Toán Clustering (Tìm nhóm)	→	K-Means, Hierarchical Clustering, DBSCAN, Gaussian Mixture Models, v.v.

Ghi Nhớ:

- Hiểu rõ bài toán trước tiên.
- Kiểm tra loại output bạn cần.
- Bắt đầu đơn giản, sau đó thử model phức tạp hơn.
- Đánh giá và so sánh hiệu năng.



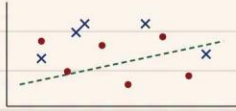
Tóm Lại:

Không có thuật toán nào là tốt nhất cho mọi bài toán.
Hãy hiểu dữ liệu, thử nhiều thuật toán khác nhau, và chọn ra thuật toán cho kết quả tốt nhất.

11. Overfitting and Underfitting

Điều này xảy ra khi model không thể **generalize** tốt trên dữ liệu mới, chưa từng thấy.
Chúng ta muốn một model hoạt động tốt trên cả **training data** và **unseen (test) data**.

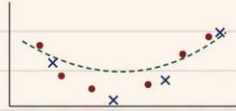
1. Underfitting (High Bias)



- Model quá đơn giản để nắm bắt pattern.
- Training error cao
- Test error cao

Model bỏ lỡ xu hướng
(quá nhiều bias)

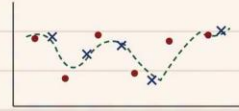
2. Good Fit (Vừa Phù Hợp)



- Model nắm bắt pattern tốt.
- Training error thấp
- Test error thấp

Model generalize tốt
(hiệu năng tốt nhất)

3. Overfitting (High Variance)



- Model quá phức tạp và khớp theo noise.
- Training error rất thấp
- Test error cao

Model học cả noise
(quá nhiều variance)

Sơ Sánh

Khía Cạnh	Underfitting (High Bias)	Good Fit (Vừa Phù Hợp)	Overfitting (High Variance)
Độ Phức Tạp Model	Quá đơn giản	Vừa đủ	Quá phức tạp
Training Error	Cao	Thấp	Rất thấp
Test / Validation Error	Cao	Thấp	Cao
Hoạt Động	Bỏ lỡ pattern quan trọng	Nắm bắt được pattern nền tảng	Khớp theo noise trong training data
Khả Năng Generalize	Kém	Tốt	Kém

Tại Sao Điều Này Xảy Ra?

- Underfitting → Model quá đơn giản, không thể học được pattern (High Bias).
- Overfitting → Model quá phức tạp, học cả noise lẫn pattern (High Variance).
- Good Fit → Cân bằng giữa Bias và Variance.

Cách Khắc Phục?

Khắc Phục Underfitting

- Dùng model phức tạp hơn
- Thêm feature
- Train thêm nhiều epoch
- Giảm regularization

Khắc Phục Overfitting

- Dùng model đơn giản hơn
- Thu thập thêm dữ liệu
- Thêm regularization (L1/L2)
- Dùng Dropout / Early Stopping

Điểm Mấu Chốt: Mục tiêu là tìm ra điểm cân bằng hoàn hảo nơi model học được pattern thực sự trong dữ liệu và hoạt động tốt trên dữ liệu mới, chưa từng thấy.

12. Model Evaluation

Model Evaluation là quá trình đo lường mức độ hoạt động của một machine learning model trên dữ liệu chưa từng thấy.

Nó giúp chúng ta chọn ra model tốt nhất và hiểu được điểm mạnh, điểm yếu của nó.

Mục tiêu: Dự đoán tốt trên dữ liệu thực tế, mới.

Tại Sao Cần Evaluate?

- Đo lường hiệu năng model.
- So sánh các model khác nhau
- Phát hiện overfitting hoặc underfitting
- Cải thiện và fine-tune model
- Xây dựng độ tin cậy cho dự đoán



Underfitting → High bias, cả 2 error đều cao

Good Fit → Test error thấp

Overfitting → Train error thấp, test error cao

Evaluation Metrics

Loại	Metric	Công Thức / Mô Tả	Phù Hợp Cho
Regression (Output Liên Tục)	MAE	$(1/n) \sum y_i - \hat{y}_i $	Chống chịu tốt với outliers
	MSE	$(1/n) \sum (y_i - \hat{y}_i)^2$	Phát năng sai số lớn
	RMSE	$\sqrt{\text{MSE}}$	Cùng đơn vị với target
	R ² Score	$1 - \text{SSres}/\text{SStot}$	Giải thích variance (0 đến 1)
Classification (Output Nhị Phân)	Accuracy	$(\text{TP} + \text{TN}) / (\text{TP} + \text{TN} + \text{FP} + \text{FN})$	Classes cân bằng
	Precision	$\text{TP} / (\text{TP} + \text{FP})$	Giảm thiểu false positives
	Recall	$\text{TP} / (\text{TP} + \text{FN})$	Giảm thiểu false negatives
	F1 Score	$2 \times (\text{P} \times \text{R}) / (\text{P} + \text{R})$	Khi classes mất cân bằng
	ROC-AUC	Diện tích dưới đường ROC (0-1)	Chất lượng phân loại tổng thể
Classification (Output Đa Lớp)	Accuracy	$(\text{TP} + \text{TN}) / (\text{TP} + \text{TN} + \text{FP} + \text{FN})$	Multi-class cân bằng
	Macro Avg	Trung bình metric mỗi class	Xử lý class imbalance
	Confusion Matrix	Bảng actual vs predicted	Phân tích lỗi chi tiết

Confusion Matrix (Binary Classification)

		Predicted	
		Positive	Negative
	Actual	TP TN	FN TN

Cross-Validation

- Chia dữ liệu thành k folds (ví dụ: k=5)
- Train trên k-1 folds, validate trên 1 fold
- Lặp lại k lần (mỗi fold dùng validate một lần)
- Lấy điểm trung bình
- → Đánh giá đáng tin cậy hơn

Những Điều Cần Chú Ý

- Đừng đánh giá dựa trên training data
- Chỉ dùng test data một lần ở cuối
- Kiểm tra overfitting
- Dùng metric phù hợp với bài toán

Tóm Lại:

Evaluation tốt cho bạn biết model của bạn generalize tốt như thế nào trên dữ liệu mới.

Hãy chọn đúng metric, validate cẩn thận, và luôn test trên dữ liệu chưa từng thấy.

13. Feature Engineering

Feature Engineering là quá trình tạo, biến đổi, và chọn lọc các feature phù hợp từ raw data để cải thiện hiệu năng model. Feature tốt → Học tốt hơn → Dự đoán tốt hơn.
Mục tiêu: Làm cho dữ liệu hữu ích hơn đối với model.

Tại Sao Nó Quan Trọng?

- Cải thiện độ chính xác của model.
- Giảm thời gian training
- Giúp model generalize tốt hơn
- Có thể giảm overfitting
- Đôi khi còn quan trọng hơn cả bản thân thuật toán!

Raw Data vs Engineered Features

Raw Data			Engineered Features			
Tuổi	Ngày sinh	Lương	Tuổi	KM	Lương(Lakh)	Nhóm Tuổi
25	12-05-1999	40.000	25	2	4.0	20-30
30	23-11-1994	60.000	30	5	6.0	30-40
45	02-07-1979	90.000	45	20	9.0	40+
...	...	...	...	...	...	...

Các Loại Feature Engineering

1. **Creation** = Tạo feature mới từ các feature có sẵn.
2. **Transformation** = Scale, encode, hoặc chuyển đổi feature.
3. **Selection** = Giữ lại feature quan trọng, loại bỏ feature không cần.
4. **Extraction** = Giảm chiều dữ liệu bằng các kỹ thuật.
5. **Interaction** = Kết hợp các feature để nắm bắt mối quan hệ.

Các Kỹ Thuật Phổ Biến

- Handling Missing Values
- Encoding Categorical Data
- Scaling Numerical Data
- Creating New Features
- Binning / Bucketing
- Feature Interaction
- Dimensionality Reduction (PCA)
- Feature Selection (Filter/Wrapper/Embedded)

1. Handling Missing Values

- Xóa dòng/cột có quá nhiều giá trị thiếu.
- Impute (điền) giá trị thiếu.
 - Mean/Median (dữ liệu số)
 - Mode (dữ liệu categorical)
 - Forward/Backward fill (time series)

2. Encoding Categorical Data

- **Label Encoding** = Dùng cho dữ liệu ordinal.
- **One-Hot Encoding** = Dùng cho dữ liệu nominal.
- **Target Encoding** = Dùng trung bình target
- **Binary Encoding** = Tiết kiệm bộ nhớ

3. Scaling Numerical Data

- **Standard Scaling** (Z-score)
 $x' = (x - \mu) / \sigma$
- **Min-Max Scaling**
 $x' = (x - x_{min}) / (x_{max} - x_{min})$

4. Creating New Features (Ví Dụ)

Kỹ Thuật	Ví Dụ	Trường Hợp Dùng
Date Features	Từ "2024-05-01" → Year, Month, Day, Weekday	Time series, Seasonality
Aggregations	Tổng số lần mua, điểm TB mỗi user	Hành vi người dùng, Giao dịch
Ratios	Chi tiêu / Thu nhập	Dữ liệu tài chính
Differences	Giá trị hiện tại - trước đó	Xu hướng, Thay đổi
Log / Power	log(Salary), sqrt(Price)	Dữ liệu lệch (skewed)

5. Feature Interaction

• Kết hợp hai hoặc nhiều feature để nắm bắt mối quan hệ. Ví dụ:



6. Các Phương Pháp Feature Selection

- **Filter:** Correlation, Chi-Square, Mutual Information
- **Wrapper:** Forward Selection, Backward Elimination
- **Embedded:** L1 (Lasso), Tree-based Importance

Pro Tips

- ✓ Hiểu rõ dữ liệu của bạn.
- ✓ Visualize trước & sau khi engineering.
- ✓ Bắt đầu đơn giản, sau đó tăng dần độ phức tạp.
- ✓ Luôn validate tác động của thay đổi.

Tóm Lại

Feature tốt biến một bài toán khó thành dễ dàng đối với model. Hãy đầu tư thời gian cho feature engineering! ★

14. Model Improvement

Model Improvement là quá trình cải thiện hiệu năng, khả năng generalize, và độ tin cậy của một machine learning model.

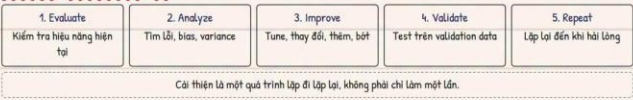
Model tốt không được xây dựng trong một lần → chúng được cải thiện từng bước.

Mục tiêu: Có được model tốt nhất, hoạt động tốt trên dữ liệu chưa từng thấy.

Tại Sao Cần Cải Thiện Model?

- Tăng độ chính xác và giảm lỗi
- Cải thiện khả năng generalize trên dữ liệu mới
- Giảm overfitting / underfitting
- Giúp model ổn định hơn
- Tiết kiệm thời gian & tài nguyên khi triển khai
- Tăng giá trị kinh doanh và độ tin cậy

Model Improvement Cycle



Các Cách Cải Thiện Model

Khía Cạnh	Cần Làm Gì	Giúp ích Như Thế Nào	Ví Dụ / Kỹ Thuật
1. Dữ Liệu Nhiều & Tốt Hơn	Thu thập thêm dữ liệu; Cải thiện chất lượng dữ liệu	Giảm overfitting; Cải thiện generalization	Lấy thêm mẫu; Loại bỏ noise, sửa nhãn
2. Feature Engineering	Tạo feature mới; Chọn feature quan trọng	Nắm bắt pattern tốt hơn; Giảm độ phức tạp	Polynomial features; Feature selection (PCA, L1)
3. Hyperparameter Tuning	Tune tham số model; Tìm thiết lập tối ưu	Hiệu năng tốt hơn; Tránh đoán thủ công	Grid Search, Random Search, Bayesian Optimization
4. Độ Phức Tạp Model	Điều chỉnh dung lượng model; Dùng độ phức tạp phù hợp	Tránh underfitting/overfitting; Cân bằng bias-variance tốt hơn	Đổi depth, trees, C, k; Regularization (L1/L2, Dropout)
5. Ensembles	Kết hợp nhiều model; Dùng phương pháp ensemble	Dự đoán ổn định hơn; Giảm variance/bias	Bagging, Boosting, Stacking; Random Forest, XGBoost
6. Lựa Chọn Thuật Toán / Model	Thử thuật toán khác nhau; Chọn ra thuật toán phù hợp	Một số model khớp dữ liệu tốt hơn; Xử lý tốt loại bài toán	Linear → Tree → Boosting; SVM → Neural Network

Chẩn Đoán Vấn Đề Trước Tiên

Underfitting (High Bias)

- Model quá đơn giản
- Giải pháp: Tăng độ phức tạp, thêm feature, train lâu hơn

Overfitting (High Variance)

- Model quá phức tạp
- Giải pháp: Thêm dữ liệu, regularization, dropout, early stopping

High Error

- Dữ liệu xấu / feature sai
- Giải pháp: Làm sạch dữ liệu, thiết kế feature tốt hơn

Các Kỹ Thuật Nền Thử

- Cross-Validation
 - Đảm bảo hiệu năng ổn định
- Learning Curves
 - Hiểu rõ bias/variance
- Regularization
 - L1, L2, Dropout
- Early Stopping
 - Dừng training khi không còn cải thiện
- Data Augmentation
 - Tạo dữ liệu mới từ dữ liệu có sẵn
- Ensembling
 - Kết hợp model yếu để tạo model mạnh

Chiến Lược Cải Thiện (Thứ Tự Ưu Tiên)

- Hiểu rõ bài toán & metric
- Cải thiện chất lượng dữ liệu
- Thực hiện feature engineering
- Tune hyperparameter
- Điều chỉnh độ phức tạp model
- Thử ensemble hoặc model nâng cao
- Validate và test trên dữ liệu chưa từng thấy

Pro Tip

- ✓ Cải thiện nhỏ ở mỗi bước dẫn đến kết quả lớn.
- ✓ Luôn validate thay đổi trước khi chuyển sang bước tiếp theo.

Tóm Lại:

Một model tốt không đến từ may mắn → mà đến từ sự cải thiện liên tục.

Hãy tiếp tục evaluate, phân tích, và lặp lại cho đến khi đạt được kết quả tốt nhất!

15. ML Model dưới dạng API

Sau khi một model được train và evaluate, chúng ta deploy nó thành một API để các ứng dụng có thể gửi dữ liệu và nhận dự đoán theo thời gian thực (real-time). Điều này giúp model có thể được sử dụng bởi các hệ thống, ứng dụng, hoặc người dùng khác.

Mục tiêu: Giúp ML model dễ truy cập, có thể mở rộng, và đáng tin cậy.

Tại Sao Nên Expose Model Thành API?

- Dự đoán real-time cho người dùng
- Dễ tích hợp với web/mobile app
- Có thể mở rộng - xử lý nhiều request cùng lúc
- Tái sử dụng trên nhiều nền tảng khác nhau
- Tách biệt ML logic và application logic

High Level Architecture



API Endpoints (Ví Dụ)

Endpoint	Method	Mô Tả	Request Body	Response
/predict	POST	Lấy dự đoán từ model	{"features": [x1, x2, x3, ...]}	{"prediction": "Yes", "probability": 0.87}
/health	GET	Kiểm tra API có đang chạy không	-	{"status": "OK"}
/info	GET	Lấy thông tin model	-	{"model": "RandomForest", "version": "1.0", "features": 20}

Các Bước Deploy Model Thành API

1. Train và lưu model
2. Xây dựng API bằng framework (FastAPI / Flask)
3. Load model vào memory
4. Tạo endpoint (ví dụ: /predict)
5. Test API trên local
6. Deploy lên server / cloud
7. Giám sát và bảo trì

Ví Dụ Luồng Xử Lý (Dự Đoán)

1. Client gửi dữ liệu đầu vào (features)
2. API nhận request tại /predict
3. API validate dữ liệu đầu vào
4. Dữ liệu được đưa vào model đã load
5. Model đưa ra dự đoán
6. API trả về dự đoán dưới dạng JSON response

Sample API Request & Response

Request (POST /predict)	Response
<pre>{ "features": [5.1, 3.5, 1.4, 0.2] }</pre>	<pre>{ "prediction": "setosa", "probability": 0.98 }</pre>

Công Cụ & Công Nghệ

Frameworks	FastAPI, Flask
Model Serialization	joblib, pickle, ONNX
Data Validation	Pydantic (FastAPI), Cerberus
Deployment	Docker, AWS (EC2, Lambda), Render, Heroku
Testing	Postman, curl, pytest

Lợi Ích

- Dự đoán real-time
- Có thể mở rộng và đáng tin cậy
- Dễ tích hợp
- Dễ bảo trì và giám sát